Mediapipe Gesture Follow (PTZ)

1. Experiment Objective

PTZ pan-tilt + car body collaboratively recognize the gesture class and follow the hand movement; MediaPipe Hands detects 21 landmarks + gesture classification, locking onto the index finger MCP as the tracking anchor.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Following	Pan-tilt tracking + coordinated body movement; the car actively turns and moves forward/back to follow
MediaPipe Hands + gesture recognition	21 landmarks + `fingersUp()` finger states + `get_gesture()` gesture-name classification
PD Control	Proportional-derivative control; a dual loop of PTZ 20 Hz + body 10 Hz
Error sharing / angular-rate feedforward / distance follow	The core mechanisms of the three-layer coordinated control (same framework as Color Follow)
Gesture-control decoupling	Gesture recognition is display-only and does not affect control output
Lost-frame tolerance	The body is frozen only after N consecutive lost frames (lost_frame_threshold=5)

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
  → gesture_follow_ptz_node
    ├── MediaPipe Hands → 21 landmarks → fingersUp() + get_gesture() gesture
classification
    ├── PTZ PD 20Hz (with body_ff_gain feedforward compensation) →
/cmd_ptz_angle
    └── Body PD 10Hz
        ├── s1 × body_share → PD → /cmd_vel.angular.z (turning)
        └── palm bbox size → distance PD → /cmd_vel.linear.x (forward/back)

The gesture recognition result is overlaid for display and does not affect the
PTZ or body control output.
```

> A MediaPipe Python environment is required (ROS2 and the workspace environment).

3. Core Topics Quick Reference

Firmware Uplink (data upstream):

Topic Name	Message Type	QoS	Description
`/camera/image_raw`	`sensor_msgs/Image`	BEST_EFFORT, KEEP_LAST depth=1, VOLATILE	Raw image stream from the ESP32 camera
`/odom`	`nav_msgs/Odometry`	BEST_EFFORT	Odometry data (published by the state estimation layer)

Firmware Downlink (command downstream):

Topic Name	Message Type	QoS	Field	Range	Description
`/cmd_ptz_angle`	`geometry_msgs/Vector3`	BEST_EFFORT, KEEP_LAST depth=1	`x` (horizontal)	-90°~90°	s1 horizontal angle
			`y` (tilt)	-90°~20°	s2 tilt angle
`/cmd_vel`	`geometry_msgs/Twist`	BEST_EFFORT, KEEP_LAST depth=1	`angular.z`	±0.12 rad/s	Turning angular velocity
			`linear.x`	±0.12 m/s	Forward/back linear velocity

ROS2 Internal (internal communication):

Topic Name	Message Type	QoS	Description
`/joint_states`	`sensor_msgs/JointState`	RELIABLE	Joint states published by the state estimation layer

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑
No-PTZ version	❌
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

2. Start Agent + Camera + Joint Model (Terminal 1)

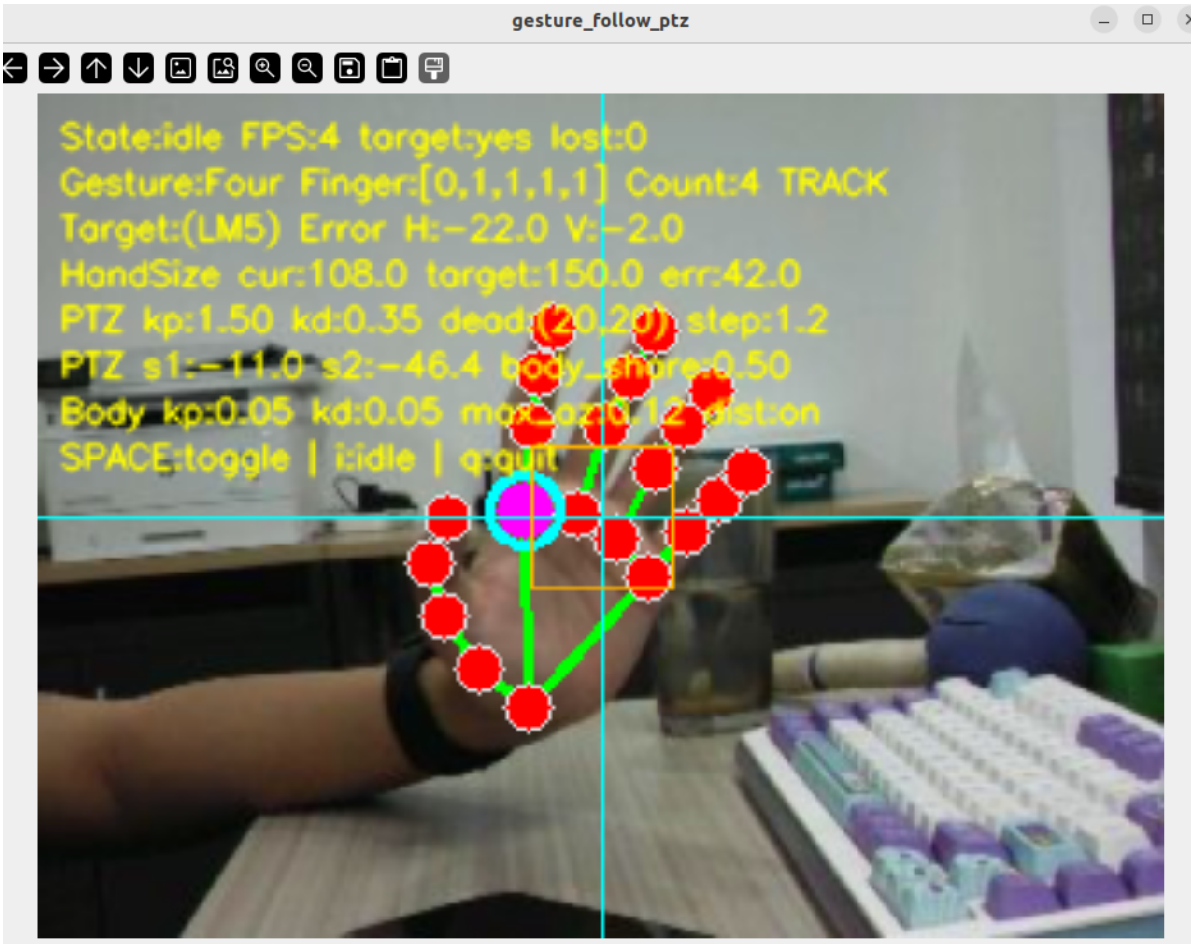
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info     | UDPv4AgentLinux.cpp | init                  | running...             | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Gesture Follow (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg gesture_follow_ptz.launch.py
```

After startup, the frame shows the `idle` state; gesture recognition runs but outputs no control.



4. Operation Instructions

(1) Workflow

The three-layer coordinated framework (PTZ 20 Hz + body 10 Hz + distance 10 Hz), with MediaPipe Hands replacing HSV detection.

Gesture detection and recognition: image scaled to 320×240 → `HandDetector(detectionCon=0.8)` → 21 landmarks + `fingersUp()` + `get_gesture()` gesture-name classification. The index finger MCP (landmark 5) is the tracking anchor.

Three-layer control:

- **PTZ PD:** `pd_kp` =1.5, `pd_kd` =0.35 (higher gains to accommodate fast hand movement)
- **Body angular velocity:** s1 × `body_share` (0.5) → PD (`angular_kp` =0.05)
- **Distance follow:** deviation between the palm bbox and `target_hand_size_px` (150 px) → PD

Gesture + follow decoupling: gesture recognition and pan-tilt/body control are fully decoupled — the recognition result is only overlaid for display and does not affect the control output. A fist pauses control but does not reset the PD state.

(2) Interaction

Key	Function
Space	idle ↔ tracking
i / I	idle
q / Q / ESC	Exit

(3) Safety and Edge Cases

- No hand / fist ≥ 5 frames → freeze the body
- Gesture detection never changes the control state
- Pan-tilt limiting [-90°, 90°] / [-90°, 20°], velocity limiting ±0.12
- Stop first on exit

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **idle state**: Gesture recognition runs but outputs no control, and the state shows `idle`
- **tracking + gesture present**: The pan-tilt locks onto the index finger MCP, the body follows, and the gesture name/finger code is displayed in real time
- **Making different gestures**: The gesture class change does not affect following; the display switches directly
- **Fist**: Control pauses but the PD is kept; it resumes when the palm opens
- **Palm lost ≥ 5 frames**: the body freezes

5. Program Analysis

> Source code paths:

- Launch file: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/gesture_follow_ptz.launch.py`
- Node file: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/gesture_follow_ptz_node.py`
- Hand detection module: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/hand_detector.py`

1. Core Node Analysis

The node `GestureFollowPtzNode` inherits from `rclpy.node.Node`, detects the 21 hand landmarks with MediaPipe Hands and recognizes the gesture type, and implements complete follow control through the PTZ PD + body dual-loop PD. The architecture is the same as `HandFollowPtzNode`, but adds a gesture classification layer and the `body_ff_gain` feedforward compensation.

Constructor `__init__` core logic:

```
def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._image_lock = threading.Lock()
    self._gesture_lock = threading.Lock()

    self._state = STATE_IDLE        # minimal state machine: idle ↔ tracking
    self._shutdown = False

    self._has_hand = False
    self._has_target = False
    self._lost_frames = 0
    self._finger_count = 0
    self._gesture_name = "None"
    self._finger_code = "[0,0,0,0,0]"
    self._target_x: Optional[float] = None
    self._target_y: Optional[float] = None
    self._target_size_px: Optional[float] = None

    self._h_deg = 0.0               # initial s1 horizontal angle
    self._p_deg = 0.0               # initial s2 tilt angle

    # PD control history state
    self._prev_err_x = 0.0
    self._prev_err_y = 0.0
    self._prev_ctrl_t = 0.0
    self._last_sx = 0.0
    self._last_sy = 0.0

    self._prev_s1 = 0.0
    self._prev_s1_eff = 0.0
    self._prev_body_t = 0.0
    self._last_body_az = 0.0

    self._prev_size_err = 0.0
    self._prev_linear_t = 0.0
```

```
    self._declare_params()
    self._load_params()

    self._h_deg = float(self._init_h_deg)
    self._p_deg = float(self._init_p_deg)

    # Initialize the MediaPipe Hands detector (with the confidence parameter)
    self._hand_det = HandDetector(detectorCon=float(self._hand_conf))

    # Image subscription (QoS: BEST_EFFORT, KEEP_LAST depth=1, VOLATILE)
    video_qos = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self._img_sub = self.create_subscription(
        Image, self._img_topic, self._on_image, video_qos
    )
    self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, 10)

    # Three independent timers: image processing, PTZ control, body control
    self._img_timer = self.create_timer(
        1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image)
    self._ptz_timer = self.create_timer(
        1.0 / max(float(self._ptz_rate), MINIMUM_TIMER_RATE_HZ), self._ptz_tick)
    self._body_timer = None
    self._rebuild_body_timer(False)

    # Initial PTZ angle publishing (configurable count and interval)
    self._init_ptz_timer = None
    self._init_ptz_remain = 0
    if self._pub_init_ptz:
        self._start_init_ptz()
```

PTZ control callback `_ptz_tick` (with the body_ff_gain feedforward compensation):

```
def _ptz_tick(self) -> None:
    if self._state != STATE_TRACKING:
        return
    with self._gesture_lock:
        tx = self._target_x
        ty = self._target_y
        has_target = self._has_target
        fc = self._finger_count

    if not has_target or fc == 0 or tx is None or ty is None:
        self._reset_ptz_pd()
        return

    icx = float(self._proc_w) * 0.5
    icy = float(self._proc_h) * 0.5
    pex = float(tx - icx)
    pey = float(ty - icy)

    cx = abs(pex) < self._db_x
```

```
    cy = abs(pey) < self._db_y
    if cx: pex = 0.0
    if cy: pey = 0.0

    n = time.perf_counter()
    dt = n - self._prev_ctrl_t if self._prev_ctrl_t > 0.0 \
         else 1.0 / max(float(self._ptz_rate), MINIMUM_TIMER_RATE_HZ)
    dt = max(dt, 1e-3)

    nex = pex / icx
    ney = pey / icy
    dx = (nex - self._prev_err_x) / dt
    dy = (ney - self._prev_err_y) / dt

    # Feedforward compensation: subtract body_ff_gain × last_body_az
    ff = self._body_ff_gain * self._last_body_az
    rx = clamp((self._kp * nex + self._kd * dx - ff) * self._max_step,
               -self._max_step, self._max_step)
    ry = clamp((self._kp * ney + self._kd * dy) * self._max_step,
               -self._max_step, self._max_step)

    a = self._step_alpha
    sx = (1.0 - a) * self._last_sx + a * rx
    sy = (1.0 - a) * self._last_sy + a * ry
    if cx: sx = 0.0; nex = 0.0
    if cy: sy = 0.0; ney = 0.0

    self._prev_err_x = nex
    self._prev_err_y = ney
    self._prev_ctrl_t = n
    self._last_sx = sx
    self._last_sy = sy

    if self._rev_h: sx = -sx
    if self._rev_p: sy = -sy

    dh = clamp(self._h_deg + sx, self._h_min, self._h_max)
    dp = clamp(self._p_deg + sy, self._p_min, self._p_max)

    sa = self._ptz_alpha
    self._h_deg = clamp((1.0 - sa) * self._h_deg + sa * dh,
                        self._h_min, self._h_max)
    self._p_deg = clamp((1.0 - sa) * self._p_deg + sa * dp,
                        self._p_min, self._p_max)

    self._pub_ptz()
```

`_ptz_tick` control chain:

```
TRACKING + has_target + not a fist
  → normalized pixel error nex, ney (zeroed in the deadband)
  → PD step = (kp × error + kd × derivative − body_ff_gain × last_body_az) × max_step
    ↑ feedforward compensation: the pan-tilt compensates in the opposite direction when the body rotates (body_ff_gain=1.3)
  → step EMA smoothing (step_alpha=0.35)
  → angle EMA smoothing (ptz_smooth_alpha=0.85)
  → clamp the angle and publish to /cmd_ptz_angle
```

2. Key Parameters

> Parameter definition file: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/gesture_follow_ptz.launch.py`

Parameter	Type	Default Value	Description
`pd_kp` / `pd_kd`	float	`1.5` / `0.35`	Pan-tilt PD gains
`ptz_control_rate`	float	`20.0` Hz	Pan-tilt control rate
`body_control_rate`	float	`10.0` Hz	Body control rate
`body_share`	float	`0.5`	Ratio of the error taken on by the body
`body_ff_gain`	float	`1.3`	Angular-rate feedforward gain
`hand_detection_confidence`	float	`0.8`	Palm detection confidence threshold
`enable_distance_follow`	bool	`true`	Enable distance following
`target_hand_size_px`	float	`150.0` px	Distance-follow target size
`distance_tolerance_hand_size_px`	float	`12.0` px	Distance tolerance deadband
`distance_kp` / `distance_kd`	float	`0.008` / `0.003`	Distance PD gains
`max_angular_z`	float	`0.12` rad/s	Maximum turning velocity
`max_linear_x`	float	`0.12` m/s	Maximum linear velocity
`lost_frame_threshold`	int	`5`	Lost-frame tolerance

6. Frequently Asked Questions

Problem	Cause	Solution
Follow + gesture double delay	MediaPipe + PTZ + body three-layer processing stacked	Reduce the resolution or reduce `ptz_control_rate`
Gesture misjudgment does not affect following	Gesture recognition is decoupled from control; display only	Normal behavior
No display on startup	The MediaPipe environment was not sourced	Run `source /opt/ros/humble/setup.bash` `source ~/microros_ws/install/setup.bash`